

**Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки**

Лабораторна робота №3

з дисципліни
«Об'єктно-орієнтоване програмування»
на тему
«Розробка інтерфейсу користувача»

Виконав:

студент групи ІМ-52
Пращук Данило Павлович
Номер в списку - 16

Перевірив:

Рекечинський Д. О.

Київ 2026

Завдання

1. Створити у середовищі розробки проект з ім'ям Lab3.
2. Написати вихідний текст програми згідно варіанту завдання.
3. Скомпілювати вихідний текст і отримати виконуваний файл програми.
4. Перевірити роботу програми. За потреби налагодити програму.
5. Проаналізувати та прокоментувати результати та вихідний текст програми.
6. Оформити звіт.

Варіанти завдань та основні вимоги

1. У звіті повинна бути схема успадкування класів – діаграма класів
2. Усі методи-обробники повідомлень, зокрема, і метод OnNotify, повинні бути функціями-членами деякого класу (класів).
3. Для вибору типу об'єкту в графічному редакторі Lab3 повинно бути вікно Toolbar з кнопками відповідно типам об'єктів.

Кнопки дублюють підпункти меню "Об'єкти". Кнопки мають бути з підказками (tooltips). Меню "Об'єкти" повинно бути праворуч меню "Файл" та ліворуч меню "Довідка".

Підпункти меню "Об'єкти" містять назви геометричних форм українською мовою. Геометричні форми згідно варіанту завдання.

4. Для вибору варіанту використовується значення $J = J_{\text{Лаб2}} + 1$, де $J_{\text{Лаб2}}$ – номер студента в журналі, який використовувався для попередньої лабораторної роботи №2.
5. Повністю повторюєте вимоги лабораторної роботи №2 від пункту 4 до пункту 7 із новим значенням J .

$$Ж = 16 + 1 = 17$$

$17 \bmod 3 \neq 0$, тому масив статичний розміру 117.

$17 \bmod 4 = 1$, тому гумовий слід - суцільна лінія червоного кольору.

Прямокутник:

$17 \bmod 2 = 1$, тому введення від центру до одного з кутів.

$17 \bmod 5 = 2$, тому відображення чорний контур з кольоровим заповненням.

$17 \bmod 6 = 5$, тому колір заповнення сірий.

Еліпс:

$17 \bmod 2 = 1$, тому введення по двох протилежних кутах охоплюючого прямокутника.

$17 \bmod 5 = 2$, тому відображення чорний контур еліпсу без заповнення.

Позначка поточного типу об'єкта:

$17 \bmod 2 = 1$, тому в заголовку вікна.

main.cpp

```
#include <QApplication>
#include <QMainWindow>
#include "mainWindow.h"

int main(int argc, char *argv[])
{
    QApplication app(argc, argv);
    MainWindow window;

    window.show();
    return app.exec();
}
```

mainWindow.cpp

```
#include "mainWindow.h"
#include <QMenu>
#include <QMenuBar>
#include <QToolBar>
#include <QIcon>
#include <QActionGroup>
#include <QMessageBox>
#include <QMouseEvent>
#include <QPaintEvent>
#include <QPainter>
#include <QRect>

MainWindow::MainWindow(QWidget *parent) : QMainWindow(parent)
{
    setWindowTitle("Lab 3");
    resize(640, 480);

    QMenuBar *menu = menuBar();
    QMenu *menuFile = menu->addMenu("&Файл");
    QAction *fileAction = menuFile->addAction("Очистити");

    toolBar = addToolBar("Панель фігур");
    toolBar->setMovable(false);

    QActionGroup *shapeGroup = new QActionGroup(this);
    shapeGroup->setExclusive(true);

    QAction *toolbarPoint = new QAction(QIcon(":/icons/point.png"), "Крапка", this);
    toolbarPoint->setCheckable(true);
    toolbarPoint->setToolTip("Інструмент: Крапка");
}
```

```

QAction *toolbarLine = new QAction(QIcon(":/icons/line.png"), "Лінія", this);
toolbarLine->setCheckable(true);
toolbarLine->setToolTip("Інструмент: Лінія");

QAction *toolbarRect = new QAction(QIcon(":/icons/rectangle.png"), "Прямокутник", this);
toolbarRect->setCheckable(true);
toolbarRect->setToolTip("Інструмент: Прямокутник (від центру)");

QAction *toolbarEllipse = new QAction(QIcon(":/icons/ellipse.png"), "Еліпс", this);
toolbarEllipse->setCheckable(true);
toolbarEllipse->setToolTip("Інструмент: Еліпс");

shapeGroup->addAction(toolbarPoint);
shapeGroup->addAction(toolbarLine);
shapeGroup->addAction(toolbarRect);
shapeGroup->addAction(toolbarEllipse);

toolBar->addAction(toolbarPoint);
toolBar->addAction(toolbarLine);
toolBar->addAction(toolbarRect);
toolBar->addAction(toolbarEllipse);

QMenu *menuObjects = menu->addMenu("&Об'єкти");
menuObjects->addAction(toolbarPoint);
menuObjects->addAction(toolbarLine);
menuObjects->addAction(toolbarRect);
menuObjects->addAction(toolbarEllipse);

QMenu *menuInfo = menu->addMenu("&Довідка");
QAction *infoAction = menuInfo->addAction("Довідкова інформація");

connect(fileAction, &QAction::triggered, this, [this]()
{
    for (int i = 0; i < shapeCount; i++) {
        delete pcshape[i];
        pcshape[i] = nullptr;
    }
    shapeCount = 0;
    redrawCanvas(); });

connect(infoAction, &QAction::triggered, this, [this]()
{ QMessageBox::information(this, "Довідка",
    "Лабораторна робота №3 «Розробка інтерфейсу користувача».<br>"
    "Варіант: 17<br>"
    "Автор: Пращук Данило Павлович ІМ-52"); });

connect(shapeGroup, &QActionGroup::triggered, this, &MainWindow::onNotify);

```

```
    toolbarPoint->setChecked(true);  
    onNotify(toolbarPoint);  
}
```

```
MainWindow::~MainWindow()  
{  
    delete currentShape;  
    for (int i = 0; i < shapeCount; i++)  
    {  
        delete pcshape[i];  
    }  
}
```

```
void MainWindow::onNotify(QAction *action)  
{  
    delete currentShape;  
    currentShape = nullptr;  
  
    if (action->text() == "Крапка")  
    {  
        currentShape = new PointShape();  
        setWindowTitle("Lab 3 - Крапка");  
    }  
    else if (action->text() == "Лінія")  
    {  
        currentShape = new LineShape();  
        setWindowTitle("Lab 3 - Лінія");  
    }  
    else if (action->text() == "Прямокутник")  
    {  
        currentShape = new RectShape();  
        setWindowTitle("Lab 3 - Прямокутник");  
    }  
    else if (action->text() == "Еліпс")  
    {  
        currentShape = new EllipseShape();  
        setWindowTitle("Lab 3 - Еліпс");  
    }  
}
```

```
void MainWindow::redrawCanvas()  
{  
    pixmap.fill(Qt::white);  
    QPainter painter(&pixmap);  
  
    int topOffset = menuBar()->height() + toolBar->height();  
    painter.setClipRect(0, topOffset, width(), height() - topOffset);  
}
```

```

for (int i = 0; i < shapeCount; i++)
{
    if (pcshape[i])
    {
        pcshape[i]->show(painter);
    }
}
update();
}

void MainWindow::mousePressEvent(QMouseEvent *event)
{
    int topOffset = menuBar()->height() + toolBar()->height();
    if (event->pos().y() < topOffset)
        return;

    if (event->button() == Qt::LeftButton && currentShape != nullptr)
    {
        if (shapeCount >= N)
            return;

        startPt = event->pos();
        endPt = event->pos();
        isDragging = true;

        if (dynamic_cast<PointShape *>(currentShape))
        {
            pcshape[shapeCount] = currentShape->clone();
            pcshape[shapeCount]->set(event->pos().x(), event->pos().y(), event->pos().x(), event->pos().y());
            shapeCount++;
            isDragging = false;
            redrawCanvas();
        }
    }
}

void MainWindow::mouseMoveEvent(QMouseEvent *event)
{
    if (isDragging)
    {
        endPt = event->pos();
        update();
    }
}

void MainWindow::resizeEvent(QResizeEvent *event)
{
    QMainWindow::resizeEvent(event);
}

```

```

    pixmap = QPixmap(size());
    redrawCanvas();
}

void MainWindow::mouseReleaseEvent(QMouseEvent *event)
{
    if (event->button() == Qt::LeftButton && isDragging)
    {
        isDragging = false;
        endPt = event->pos();

        if (currentShape && shapeCount < N)
        {
            pcshape[shapeCount] = currentShape->clone();
            pcshape[shapeCount]->set(startPt.x(), startPt.y(), endPt.x(), endPt.y());
            shapeCount++;
            redrawCanvas();
        }
    }
}

```

```

void MainWindow::paintEvent(QPaintEvent *event)
{
    QMainWindow::paintEvent(event);
    QPainter painter(this);

    int topOffset = menuBar()->height() + toolBar->height();
    QRect drawingArea(0, topOffset, width(), height() - topOffset);
    painter.setClipRect(drawingArea);

    painter.drawPixmap(0, 0, pixmap);

    if (isDragging && currentShape)
    {
        currentShape->set(startPt.x(), startPt.y(), endPt.x(), endPt.y());
        currentShape->rubber(painter);
    }
}

```

mainwindow.h

```

#ifndef MAINWINDOW_H
#define MAINWINDOW_H

#include <QMainWindow>
#include <QPixmap>
#include "shape.h"

```

```

class QPaintEvent;
class QMouseEvent;
class QResizeEvent;
class QToolBar;

class MainWindow : public QMainWindow
{
    Q_OBJECT

public:
    MainWindow(QWidget *parent = nullptr);
    ~MainWindow();

protected:
    void paintEvent(QPaintEvent *event) override;
    void mousePressEvent(QMouseEvent *event) override;
    void mouseMoveEvent(QMouseEvent *event) override;
    void mouseReleaseEvent(QMouseEvent *event) override;
    void redrawCanvas();
    void resizeEvent(QResizeEvent *event) override;

private slots:
    void onNotify(QAction *action);
private:
    bool isDragging = false;
    QPoint startPt;
    QPoint endPt;
    QToolBar *toolBar;

    static const int N = 117;
    Shape* currentShape = nullptr;
    Shape *pcshape[N] = {nullptr};
    int shapeCount = 0;
    QPixmap pixmap;

};

#endif

```

shape.cpp

```

#include "shape.h"
#include <cmath>
#include <QPen>
#include <QColor>

Shape::Shape() : xs1(0), ys1(0), xs2(0), ys2(0) {}

```

```

void Shape::set(long x1, long y1, long x2, long y2)
{
    xs1 = x1;
    ys1 = y1;
    xs2 = x2;
    ys2 = y2;
}

void Shape::rubber(QPainter &painter)
{
    painter.setPen(QPen(Qt::red, 1, Qt::SolidLine));
    painter.setBrush(Qt::NoBrush);
}

void PointShape::show(QPainter &painter)
{
    painter.setPen(QPen(Qt::black, 6, Qt::SolidLine, Qt::RoundCap));
    painter.drawPoint(xs1, ys1);
}
Shape *PointShape::clone() const { return new PointShape(*this); }

void LineShape::show(QPainter &painter)
{
    painter.setPen(QPen(Qt::black, 2, Qt::SolidLine));
    painter.drawLine(xs1, ys1, xs2, ys2);
}
void LineShape::rubber(QPainter &painter)
{
    painter.setPen(QPen(Qt::red, 1, Qt::SolidLine));
    painter.drawLine(xs1, ys1, xs2, ys2);
}
Shape *LineShape::clone() const { return new LineShape(*this); }

void RectShape::show(QPainter &painter)
{
    painter.setPen(QPen(Qt::black, 2, Qt::SolidLine));
    painter.setBrush(QColor(128, 128, 128));

    int dx = std::abs(xs2 - xs1);
    int dy = std::abs(ys2 - ys1);
    QRect previewRect(xs1 - dx, ys1 - dy, 2 * dx, 2 * dy);
    painter.drawRect(previewRect.normalized());
}
void RectShape::rubber(QPainter &painter)
{

```

```

    painter.setPen(QPen(Qt::red, 1, Qt::SolidLine));
    painter.setBrush(Qt::NoBrush);

    int dx = std::abs(xs2 - xs1);
    int dy = std::abs(ys2 - ys1);
    QRect previewRect(xs1 - dx, ys1 - dy, 2 * dx, 2 * dy);

    painter.drawRect(previewRect.normalized());
}
Shape *RectShape::clone() const { return new RectShape(*this); }

void EllipseShape::show(QPainter &painter)
{
    painter.setPen(QPen(Qt::black, 2, Qt::SolidLine));
    painter.setBrush(Qt::NoBrush);

    QRect previewRect(QPoint(xs1,ys1), QPoint(xs2,ys2));
    painter.drawEllipse(previewRect.normalized());
}
void EllipseShape::rubber(QPainter &painter)
{
    painter.setPen(QPen(Qt::red, 1, Qt::SolidLine));
    painter.setBrush(Qt::NoBrush);
    QRect previewRect(QPoint(xs1,ys1), QPoint(xs2,ys2));
    painter.drawEllipse(previewRect.normalized());
}
Shape *EllipseShape::clone() const { return new EllipseShape(*this); }

```

shape.h

```

#ifndef SHAPE_H
#define SHAPE_H

#include <QPainter>
#include <QRect>
#include <QPoint>

class Shape
{
protected:
    long xs1, ys1, xs2, ys2;

public:
    Shape();
    virtual ~Shape() = default;

```

```
void set(long x1, long y1, long x2, long y2);  
virtual void show(QPainter &painter) = 0;  
virtual void rubber(QPainter &painter);  
virtual Shape* clone() const = 0;  
};
```

```
class PointShape : public Shape  
{  
public:  
    void show(QPainter &painter) override;  
    Shape* clone() const override;  
};
```

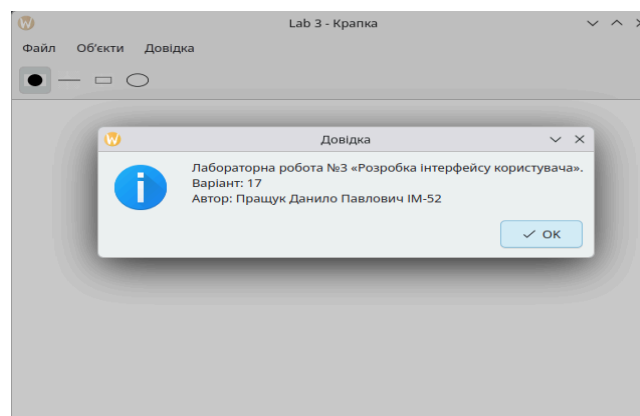
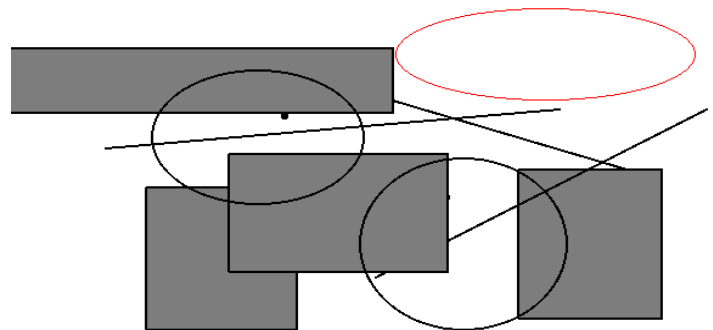
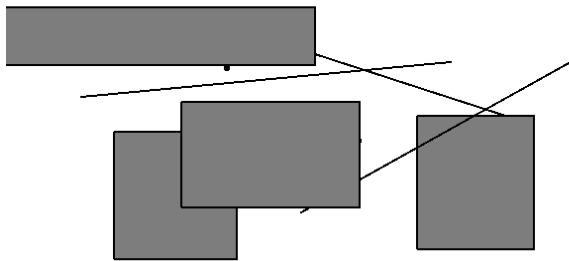
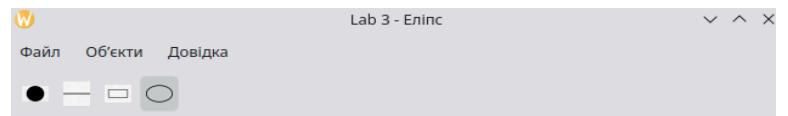
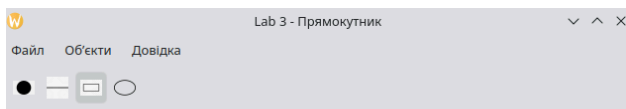
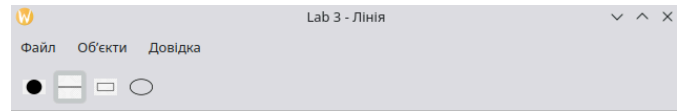
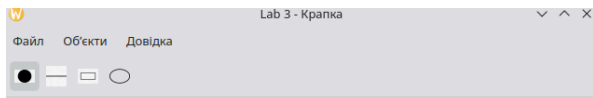
```
class LineShape : public Shape  
{  
public:  
    void show(QPainter &painter) override;  
    void rubber(QPainter &painter) override;  
    Shape* clone() const override;  
};
```

```
class RectShape : public Shape  
{  
public:  
    void show(QPainter &painter) override;  
    void rubber(QPainter &painter) override;  
    Shape* clone() const override;  
};
```

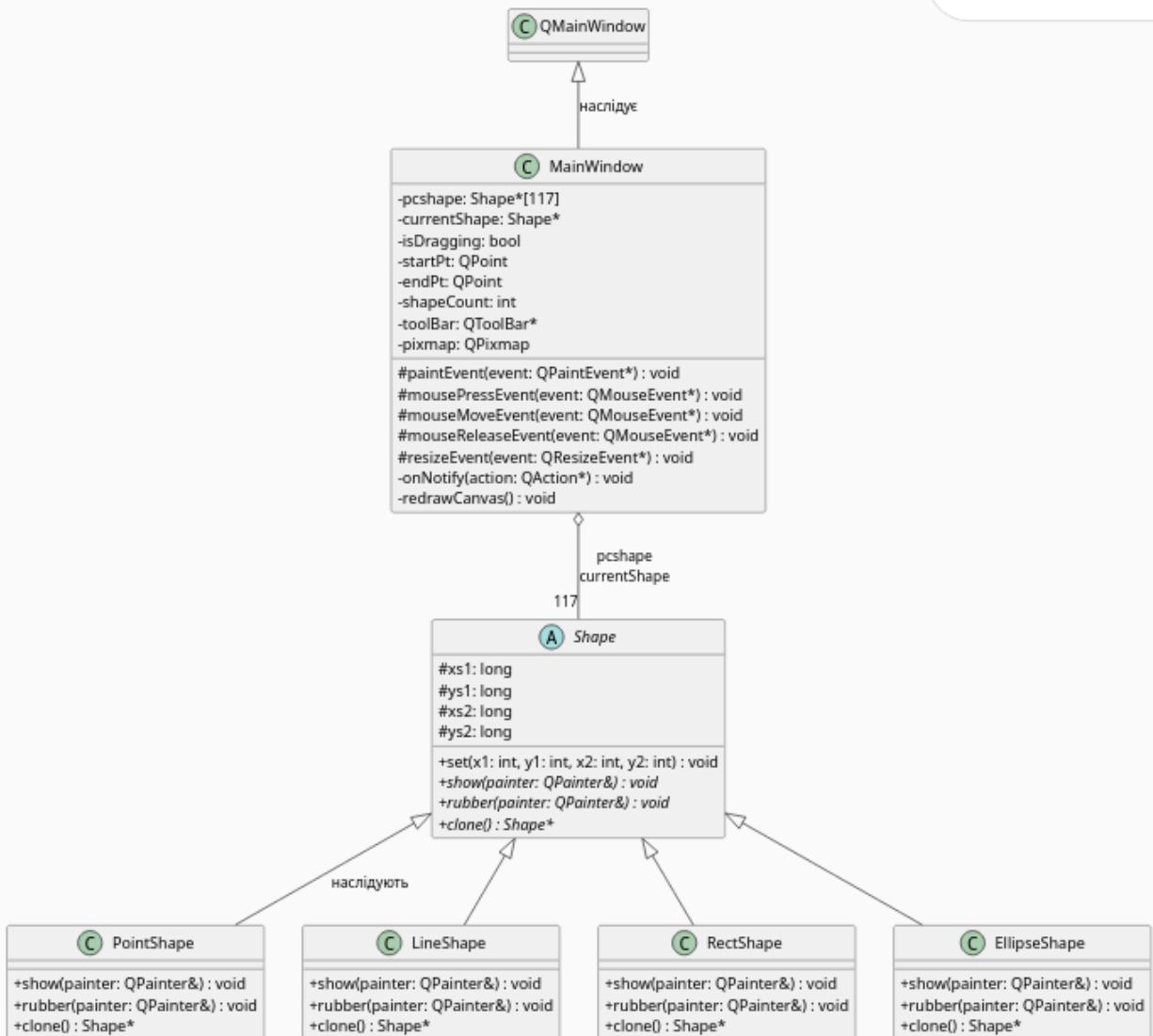
```
class EllipseShape : public Shape  
{  
public:  
    void show(QPainter &painter) override;  
    void rubber(QPainter &painter) override;  
    Shape* clone() const override;  
};
```

```
#endif
```

Скріншоти



Діаграма залежностей класів



Висновки:

Під час виконання лабораторної роботи №3 було розроблено графічний інтерфейс користувача на базі фреймворку Qt та мови C++, у якому за допомогою інкапсуляції, успадкування та поліморфізму реалізовано ієрархію фігур із віртуальними методами `show`, `rubber` і `clone`.

Для вибору геометричних форм створено панель інструментів `QToolBar` із власними іконками та підказками (tooltips), яка дублює підпункти меню «Об'єкти», розташованого між меню «Файл» та «Довідка».

Синхронізацію стану активного інструмента забезпечено за допомогою єдиних об'єктів `QAction` у складі `QActionGroup`, а обробку вибору форми інкапсульовано у функції-члені класу `onNotify`.

Усі вимоги варіанта 17 — включно зі збереженням об'єктів у статичному масиві на 117 елементів, відображенням поточної фігури в заголовку вікна, червоним суцільним «гумовим» слідом, прямокутником від центру із сірим заповненням та незаповненим еліпсом — виконано в повному обсязі.